Testes de Software

Análise de Eficácia de Testes com Teste de Mutação

Biblioteca de operações matemáticas
Jest e StrykerJS

Aluno: Matheus Fernandes de Oliveira

Matrícula: 773220

Repositório:
https://github.com/matheus-0063/operacoes-mutante

13/10/2026

1. Análise inicial e método

O experimento utiliza as 50 operações do projeto base. A suíte original foi preservada; 55 novos casos foram adicionados em um arquivo complementar. O código de produção permaneceu igual nas duas execuções e foram avaliados os mesmos 213 mutantes, sem exclusões de operadores.

Indicador	Inicial	Final
Testes aprovados	50	105
Cobertura de linhas	98,64%	100,00%
Cobertura de instruções	85,41%	100,00%
Cobertura de ramificações	58,82%	100,00%
Cobertura de funções	100,00%	100,00%
Pontuação de mutação bruta	73,71%	97,65%

A cobertura inicial de linhas era alta, mas a suíte deixou 44 mutantes sobreviventes e 12 sem cobertura. Executar uma linha não comprova a eficácia da asserção: as entradas originais omitiam fronteiras, resultados falsos, arrays vazios e valores que revelam erros de cálculo.

Ambiente: Node v24.8.0, Jest 29.7.0 e StrykerJS 9.6.1. Comandos: npm run coverage:inicial, npm run mutate:inicial, npm run coverage e npm run mutate. O módulo foi configurado como CommonJS, conforme require/module.exports já utilizados pelo código.

2. Mutantes críticos e soluções

Caso 1 - Mutante 177: comparação na igualdade

Na linha 104, **a > b** foi substituído por **a >= b**. O teste inicial usava (10, 5), que resulta em true nas duas versões. Faltava verificar a fronteira a === b.

```
expect(op.isMaiorQue(5, 5)).toBe(false);
```

O novo caso produz false no original e true no mutante. O relatório final registra **Killed** para o mesmo mutante que inicialmente tinha estado **Survived**.



Figura: detalhe do mutante 177 no relatório inicial do Stryker.

Caso 2 - Mutante 193: mediana sem ordenação

Na linha 109, foi removida a chamada **.sort((a, b) => a - b)**. O teste original fornecia [1, 2, 3, 4, 5], que já estava ordenado. A posição central continuava correta mesmo quando o algoritmo ignorava a ordenação.

```
expect(op.medianaArray([3, 1, 2])).toBe(2);
expect(op.medianaArray([9, 1, 5, 3])).toBe(4);
```

A entrada desordenada [3, 1, 2] tem mediana 2, mas o mutante retorna 1. O segundo caso cobre a média dos dois elementos centrais de um array par. Outros casos verificam ordenação numérica, valores negativos e preservação do array original. Estado final: **Killed**.



Figura: detalhe do mutante 193 no relatório inicial do Stryker.

Caso 3 - Mutante 156: fator de conversão oculto pelo zero

Na linha 93, **celsius * 9 / 5** passou a **celsius / 9 / 5**. O teste original usava 0 °C: o termo multiplicativo permanece zero e ambas as versões retornam 32 °F.

```
expect(op.celsiusParaFahrenheit(100))
  .toBeCloseTo(212, 10);
```

Com 100 °C, o resultado correto é 212 °F e o mutante retorna aproximadamente 34,22 °F. A entrada torna o fator de conversão observável. Foram acrescentados 25 °C, -40 °C e casos não nulos na conversão inversa. Estado final: **Killed**.

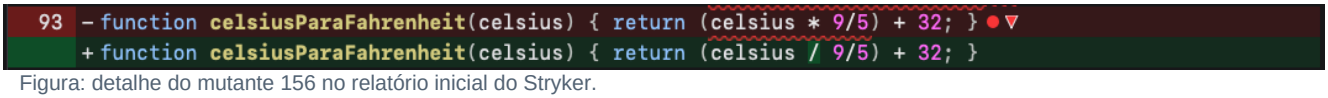


Figura: detalhe do mutante 156 no relatório inicial do Stryker.

Estratégia dos novos casos de teste

Além dos três exemplos, a suíte complementar verifica mensagens específicas de erro, arrays vazios, resultados booleanos falsos, números compostos e valores nas fronteiras das comparações. Para a mediana, foram usados arrays pares e ímpares, valores negativos e números de dois dígitos, com asserção de que a entrada permanece inalterada.

Os valores esperados foram definidos a partir do comportamento matemático e do contrato das operações. Em conversões com ponto flutuante, toBeCloseTo usa precisão de dez casas; nos demais exemplos, toBe exige o resultado específico. Isso permite identificar alterações que a execução de uma única entrada nominal não revelava.

3. Resultados finais e limitação da meta

A pontuação bruta passou de **73,71%** para **97,65%**, ganho de **23.94 pontos percentuais**. Na fase final, 205 mutantes foram mortos por asserções, 3 foram detectados por timeout, 5 sobreviveram e nenhum ficou sem cobertura. Os 105 testes passaram e a cobertura atingiu 100% nas quatro métricas.

O Stryker inclui Killed e Timeout no numerador: **(205 + 3) / 213 = 97,65%**. Os timeouts são apresentados separadamente; não são mortes por asserção.

A meta bruta superior a 98% não foi alcançada. Os cinco sobreviventes abaixo são equivalentes nos domínios usuais das funções. Nenhum foi desativado para aumentar artificialmente o resultado.

IDs / função	Justificativa da equivalência
28, 29, 30 e 32 / fatorial	As mutações removem total ou parcialmente o retorno antecipado para 0 e 1. O acumulador já inicia em 1 e o laço começa em 2; sem o retorno antecipado, a saída continua sendo 1. Para os demais inteiros não negativos, o cálculo permanece igual.
135 / produtoArray	Sem o retorno antecipado de [], reduce com acumulador inicial 1 retorna o mesmo 1. Para arrays não vazios com reduce padrão, ambos os caminhos executam a mesma redução.

As justificativas foram apoiadas por 171 entradas de fatorial por mutante (0 a 170) e 6 arrays para produtoArray. Essas amostras confirmam os exemplos; a equivalência decorre da análise do fluxo e dos elementos neutros, dentro dos domínios declarados.

O cálculo complementar, descontando somente esses cinco equivalentes, é **208 / (213 - 5) = 100%**. Esse valor ajustado não substitui o score bruto do Stryker. Mantendo o código e o conjunto de mutantes, o teto bruto no domínio considerado é 97,65%. A interpretação da eficácia considera, portanto, tanto a pontuação bruta quanto as justificativas de equivalência apresentadas.

4. Conclusão

A cobertura mede execução; a mutação avalia a capacidade de detectar alterações de comportamento. A evolução observada veio de entradas que expõem falhas reais: igualdade, valores não nulos, arrays desordenados, casos de erro e retornos booleanos falsos. A análise dos equivalentes também demonstra que um sobrevivente exige interpretação: ele pode revelar uma lacuna nos testes ou uma transformação sem diferença observável no domínio da função.

Referências e evidências

StrykerJS, configuração: stryker-mutator.io/docs/stryker-js/configuration/
Operadores de mutação: stryker-mutator.io/docs/mutation-testing-elements/supported-mutators/
Evidências locais: reports/resumo.json e reports/mutation/{inicial,final}/mutation.json. Execuções realizadas em 13/10/2026.